ОТЧЁТ

по лабораторной работе №19

«Микросервисная архитектура Python-приложений»

Вариант 20. Система управления задачами

1. Цель работы

Научиться разделять систему на независимые сервисы и передавать событие смены статуса задачи в сервис уведомлений.

2. Постановка индивидуального задания

Вариант 20. Сервисы: пользователи, задачи и уведомления. При изменении статуса задачи нужно отправить событие.

3. Краткие теоретические сведения

Микросервис отвечает за свою область и общается с другими по сети, а не общим объектом в памяти. Сервис пользователей не хранит задачи, сервис задач не хранит журнал уведомлений.

Смена статуса — событие. Сервис задач отправляет POST с номером, названием, новым статусом и исполнителем. Сервис уведомлений принимает событие и сохраняет его. Если уведомления недоступны, задача всё равно меняет статус и помечает, что событие не ушло.

4. Описание программы

Три приложения FastAPI: users, tasks и notifications. Демонстрация создаёт пользователя Иванова, задачу «Проверить канал» и переводит её в статус «в работе». В журнале уведомлений появляется событие с тем же исполнителем и статусом.

Тест подменяет HTTP-вызов обращением к приложению уведомлений в том же процессе и проверяет, что событие «готово» дошло до получателя.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — demo.py

```
"""Последовательный запуск трёх сервисов и смена статуса задачи."""

import sys

import httpx
```

```
import uvicorn

from notify_app import create_app as notify_app
from tasks_app import create_app as tasks_app
from users_app import create_app as users_app

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")



def main():
    print("Сервисы: users :8019, tasks :8020, notifications :8021")
    print("Демонстрация идёт через TestClient-совместимые вызовы внутри процесса")
    print("и через HTTP, если серверы уже подняты отдельными командами.")
    users = users_app()
    tasks = tasks_app("http://notify.local")
    notes = notify_app()
    from fastapi.testclient import TestClient

    users_client = TestClient(users)
    notes_client = TestClient(notes)

    class LocalNotify:
        def post(self, url, json, timeout):
            response = notes_client.post("/events", json=json)
            response.raise_for_status()
            return response

    import tasks_app as tasks_module
    original = tasks_module.httpx.post
    tasks_module.httpx.post = LocalNotify().post
    tasks = tasks_app("http://notify.local")
    tasks_client = TestClient(tasks)
    try:
        user = users_client.post("/users", json={"name": "Иванов", "email": "ivan@example.com"}).json()
        print("Пользователь:", user)
        task = tasks_client.post("/tasks", json={"title": "Проверить канал", "assignee":
user["name"]}).json()
        print("Задача:", task)
        updated = tasks_client.post(f"/tasks/{task['id']}/status", json={"status": "в работе"}).json()
        print("После смены статуса:", updated)
        print("События уведомлений:", notes_client.get("/events").json())
    finally:
        tasks_module.httpx.post = original
    print("Цепочка users → tasks → notifications выполнена.")



if __name__ == "__main__":
    main()
```

Листинг — notify_app.py

```
"""Сервис уведомлений. Принимает событие смены статуса задачи."""

from fastapi import FastAPI
from pydantic import BaseModel



class Event(BaseModel):
    task_id: int
    title: str
    status: str
    assignee: str



def create_app():
    app = FastAPI(title="notifications")
    events = []

    @app.get("/health")
    def health():
        return {"service": "notifications", "events": len(events)}
```

```
    @app.post("/events", status_code=202)
    def receive(event: Event):
        events.append(event.model_dump())
        return {"accepted": True}

    @app.get("/events")
    def list_events():
        return events

    return app


app = create_app()
```

Листинг — tasks_app.py

```
"""Сервис задач. При смене статуса отправляет событие в сервис уведомлений."""

import httpx
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel


class TaskIn(BaseModel):
    title: str
    assignee: str


class StatusIn(BaseModel):
    status: str


def create_app(notify_url="http://127.0.0.1:8021"):
    app = FastAPI(title="tasks")
    tasks = {}
    seq = {"n": 1}

    @app.get("/health")
    def health():
        return {"service": "tasks"}

    @app.post("/tasks", status_code=201)
    def create_task(payload: TaskIn):
        item = {
            "id": seq["n"],
            "title": payload.title,
            "assignee": payload.assignee,
            "status": "новая",
        }
        seq["n"] += 1
        tasks[item["id"]] = item
        return item

    @app.get("/tasks")
    def list_tasks():
        return list(tasks.values())

    @app.post("/tasks/{task_id}/status")
    def change_status(task_id: int, payload: StatusIn):
        item = tasks.get(task_id)
        if item is None:
            raise HTTPException(404, "задача не найдена")
        item["status"] = payload.status
        event = {
            "task_id": item["id"],
            "title": item["title"],
            "status": item["status"],
            "assignee": item["assignee"],
        }
        try:
            httpx.post(f"{notify_url}/events", json=event, timeout=2)
```

```
            item["event"] = "отправлено"
        except httpx.HTTPError:
            item["event"] = "сервис уведомлений недоступен"
        return item

    return app


app = create_app()
```

Листинг — users_app.py

```
"""Сервис пользователей."""

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel


class UserIn(BaseModel):
    name: str
    email: str


def create_app():
    app = FastAPI(title="users")
    users = {}
    seq = {"n": 1}

    @app.get("/health")
    def health():
        return {"service": "users"}

    @app.post("/users", status_code=201)
    def create_user(payload: UserIn):
        item = {"id": seq["n"], "name": payload.name, "email": payload.email}
        seq["n"] += 1
        users[item["id"]] = item
        return item

    @app.get("/users/{user_id}")
    def get_user(user_id: int):
        item = users.get(user_id)
        if item is None:
            raise HTTPException(404, "пользователь не найден")
        return item

    return app


app = create_app()
```

Листинг — tests\test_flow.py

```
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from fastapi.testclient import TestClient

import tasks_app as tasks_module
from notify_app import create_app as notify_app
from tasks_app import create_app as tasks_app
from users_app import create_app as users_app


def test_status_event_reaches_notifications():
    notes = TestClient(notify_app())

    def fake_post(url, json, timeout):
        return notes.post("/events", json=json)
```

```
    original = tasks_module.httpx.post
    tasks_module.httpx.post = fake_post
    try:
        tasks = TestClient(tasks_app())
        created = tasks.post("/tasks", json={"title": "Отчёт", "assignee": "Петрова"})
        assert created.status_code == 201
        changed = tasks.post(f"/tasks/{created.json()['id']}/status", json={"status": "готово"})
        assert changed.status_code == 200
        events = notes.get("/events").json()
        assert events[0]["status"] == "готово"
        assert events[0]["assignee"] == "Петрова"
    finally:
        tasks_module.httpx.post = original


def test_user_created():
    client = TestClient(users_app())
    response = client.post("/users", json={"name": "Иванов", "email": "a@b.c"})
    assert response.status_code == 201
    assert client.get(f"/users/{response.json()['id']}").json()["name"] == "Иванов"
```

6. Примеры работы программы

Снимок показывает цепочку пользователь → задача → событие уведомления.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python19> python demo.py

Сервисы: users :8019, tasks :8020, notifications :8021
Демонстрация идёт через TestClient-совместимые вызовы внутри процесса
и через HTTP, если серверы уже подняты отдельными командами.
Пользователь: {'id': 1, 'name': 'Иванов', 'email': 'ivan@example.com'}
Задача: {'id': 1, 'title': 'Проверить канал', 'assignee': 'Иванов', 'status': 'новая'}
После смены статуса: {'id': 1, 'title': 'Проверить канал', 'assignee': 'Иванов', 'status': 'в работе', 'event': 'от
правлено'}
События уведомлений: [{'task_id': 1, 'title': 'Проверить канал', 'status': 'в работе', 'assignee': 'Иванов'}]
Цепочка users → tasks → notifications выполнена.
```

Рисунок 1 – Смена статуса задачи и событие в сервисе уведомлений

7. Ответы на контрольные вопросы

1. Почему это не один модуль?

У пользователей, задач и уведомлений разные причины меняться. Отдельный сервис можно развернуть и заменить, не трогая остальные.

2. Что является событием?

Факт смены статуса. В нём есть задача, новый статус и исполнитель, но нет всей базы задач.

3. Что будет, если сервис уведомлений недоступен?

Статус задачи сохранится, а в ответе будет отметка, что событие не отправлено. Учёт задач не зависит от рассылки.

4. Как сервисы находят друг друга?

Сервис задач знает адрес сервиса уведомлений. В демонстрации это локальные порты 8019, 8020 и 8021.

5. Что проверяет тест?

Что после смены статуса в журнале уведомлений есть событие с ожидаемым статусом и исполнителем.

8. Вывод

В лабораторной работе №19 система задач варианта 20 разделена на сервисы пользователей, задач и уведомлений. Смена статуса публикует событие, и сервис уведомлений его принимает. Отказ уведомлений не отменяет саму смену статуса.